Measuring the Impact of Index Types and Tuning Settings on Query Execution Plans in PostgreSQL and MariaDB

An access-path study across two independently written optimisers

Md. Imtiaj Alam Sajin

26-94090-2 | 26-94090-2@student.aiub.edu

Supervisor

Dr. Ashraf Uddin

MOTIVATION

A decision nobody sees, and advice nobody measured

Scan the table, or use an index, and if so which one.

One decision per query, invisible to the user, worth up to two orders of magnitude in runtime.

The literature attributes bad plans to cardinality misestimation. That work targets join ordering. The single-relation access-path decision has not been isolated the same way.

Four widely recommended practices

- Extended statistics for correlated columns
- Lower random_page_cost on SSD storage
- Add a BRIN index, it costs almost nothing
- Enable Multi-Range Read in MariaDB

Every one is sound in the case it was written for. This study measures where each stops being sound.

Two engines chosen (Postgres vs MariaDB)

A single-system study cannot separate a property of cost-based access-path selection from an artefact of one implementation.

Both systems run identical generated data, identical queries, identical metrics and a matched 128 MB buffer pool.

METRICS

Two measures, deliberately kept apart

R = time of the plan chosen / time of the fastest plan available

Access-path regret measures the decision. The comparison is against plans the engine really had, forced one at a time, so a slow engine is not penalised for being slow.

q-error measures the estimate: the factor by which the predicted row count misses the truth. Reported separately throughout, because the paper's main negative result is that the two come apart.

1.0

the engine made the best choice available to it

23,480

measurements across both engines

EXPERIMENTAL DESIGN

Symmetric, controlled, and seeded

Factor	Variations / Levels
Data Distributions	11 Datasets (Skew: Zipf 0.0–1.5, Dependency: 0–1.0, Clustering: 0–1.0)
Index Schemes	10 Configurations (None, B-tree, Hash, BRIN, Composite, All-Indexes)
Table Scales	7 Scales (1M to 10M rows)
Tuning Parameters	16 Settings (random_page_cost 1.0–4.0, Extended Stats ON/OFF)
Execution Matrix	23,480 total query runs in PostgreSQL 17.1 & MariaDB 10.4

	Queries Executed	
Table size	PostgreSQL	MariaDB
1M	2,398	1,232
1.25M	654	336
1.5M	654	336
2M	654	336
3M	654	336
5M	654	336
10M	2,398	1,232
Configuration sweeps	6,230	5,040
Total	**14,296**	**9,184**

10 index configurations, 4 query families, 8 selectivity targets. Seven runs per query, first discarded. Parallelism and JIT off.

RESULT 1

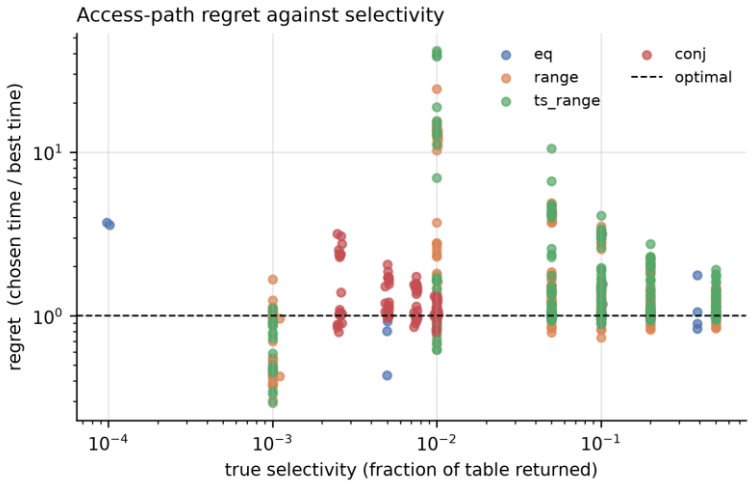
Selection quality differs by an order of magnitude

Engine	Queries	Chose badly	Median regret	Worst case
PostgreSQL	130	2.3%	1.01	1.39x
MariaDB	153	66.0%	1.34	7.91x

Read the last two columns together

At 1 million rows, PostgreSQL chose a materially worse access path in only 2.3% of queries, with a median regret of just 1%.

MariaDB, however, made such choices in 66% of queries, and a worst case of 7.91 times slower. So the important finding is that the two optimizers have very different failure profiles."



WHY THE TWO ENGINES DIFFER

The gap is the plans available, not the care taken

What the engine chose	PostgreSQL	MariaDB
Bitmap scan (sorts rows into disk order)	346	7
Direct index lookup	19	301
Gave up, scanned the whole table	9	66

A bitmap scan puts a ceiling on how bad a mistake can be, because even a wrong choice still reads the disk in order.

MariaDB has no ceiling. A mistaken index scan becomes random single-row fetches.

Counted over the free-choice configuration, all four query families, 1,000,000 rows.

This is one design decision, not a tuning gap. It is why PostgreSQL errs rarely and mildly, and MariaDB errs often and sometimes badly.

THE MAIN FINDING

The estimates were right. The decisions were wrong.

98.2%

of the times the database chose a slower plan,
it had the row count essentially correct

113 cases examined. Median estimation error 1.017, where a perfect estimate is 1.000.

Why this matters

The standard explanation for bad query plans is bad row estimates.

Here the estimates were fine. So better statistics cannot fix this, and most tuning advice is aimed at statistics.

The fault is in how the database compares the costs of the plans it already understands.

THE BIGGEST EFFECT WE MEASURED

Adding an index made the query slower

194x

slower, and the only thing that changed was that a BRIN index existed next to the B-tree

0.34 ms becomes 65.95 ms

Bad choices: 2.3% becomes 73.3%

The cause, in one sentence

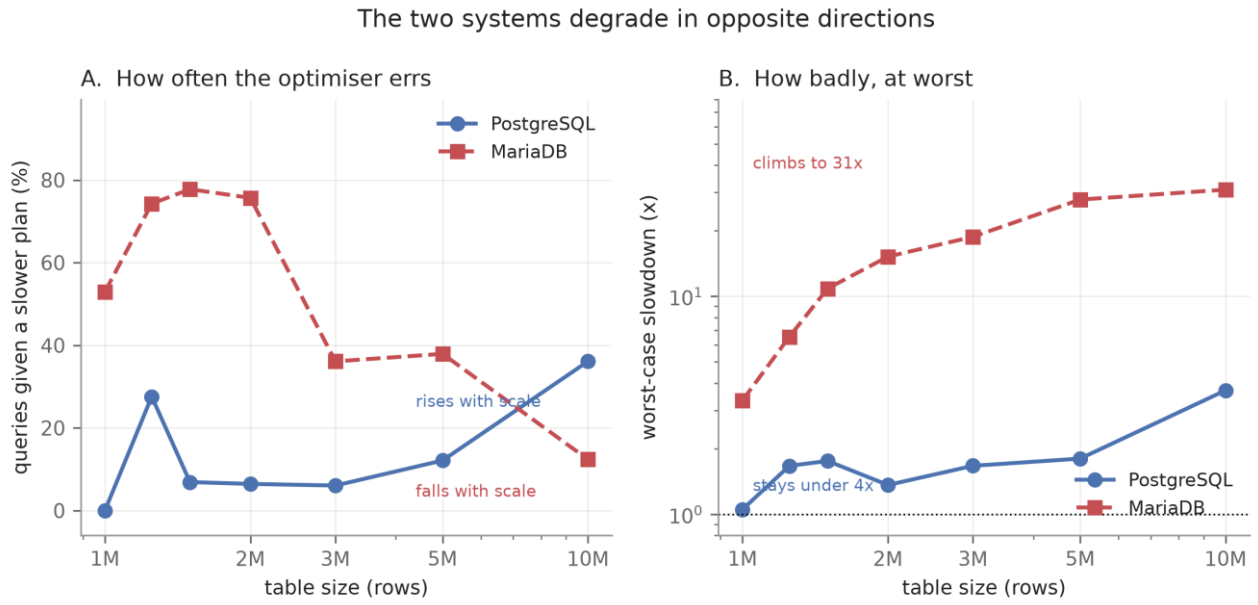
When two indexes can answer the same query, PostgreSQL keeps whichever is cheaper to read, and ignores the work the query then does on the table.

A BRIN index is tiny, so it always wins that comparison. The index it discards is the fast one.

Only while the table still fits in the buffer pool, which is exactly the size at which BRIN is recommended as free.

HOW THEY FAIL AS DATA GROWS

The two engines fail in opposite directions



Left: how often it errs. Right: how badly, at worst.

PostgreSQL

gets wrong more often as tables grow, 0% to 36%, but never by more than 3.7x.

MariaDB

gets wrong less often, 53% down to 12%, but its worst case climbs to 30.9x.

Wrong often but never badly is easier to run than wrong rarely but catastrophically.

Seven table scales, 1 to 10 million rows. Each engine judged only against plans the other could also have formed.

THE PATTERN

Four recommended practices, all four backfire

	WHAT IT PROMISES	WHAT WE MEASURED
Extended statistics for correlated columns	Fixes the estimate 108x	**Query up to 2.7x slower**
Lower random_page_cost because the disk is an SSD	Default is already near optimal	**Lowering it 2.3x worse**
Add a BRIN index it is small and cheap	Costs almost no space	**Bad choices 2.3% to 73.3%**
Enable Multi-Range Read MariaDB, ships switched off	The mechanism does engage	**Workload 7% slower**

In three of four the advice delivers exactly what it promises, and the plan gets worse anyway.

WHAT WE RECOMMEND

What to do, and what not to do

DO

+ **Measure the whole workload, not one query**
  A 24x tuning effect vanished across the workload.

+ **Test more than two table sizes**
  Frequency and severity moved in opposite directions.

+ **Report how often and how badly, separately**
  One number hides which engine is safer to run.

+ **Keep shipped defaults until measured otherwise**
  The default random_page_cost was within 6% of optimal.

DO NOT

– **Add a BRIN index next to a B-tree**
  Same column, table in memory: bad choices 2.3% to 73.3%, worst 194x.

– **Add extended statistics for a faster plan**
  Estimate improved 108x, query 1.3x to 2.7x slower.

– **Lower random_page_cost for an SSD**
  At 1.0 the workload was 2.3x worse than the default.

– **Enable MariaDB MRR at its default buffer**
  7% slower; helps only at 8 MB and only on narrow queries.

All four were reasonable advice. None of it shipped with the conditions under which it stops being true.